HoT Skill Assessment – Explanation

Name: R Cheran
Reg No: RA2311026050276

Introduction: Compiler design consists of multiple phases such as lexical analysis, syntax analysis, semantic analysis, and code generation. Each phase produces raw data that is not directly useful for evaluating code quality. Therefore, feature engineering is required to convert raw compiler outputs into meaningful indicators. This project focuses on applying High Order Thinking Skills (HOTS) to transform raw compiler data into measurable metrics using Python and Pandas. Problem 1: Token Density Indicator analyzes lexical tokens to measure code complexity. Tokens are grouped by file and line, and the average tokens per line is computed. A threshold is applied to identify complex code. High density indicates reduced readability and maintainability. Problem 2: Identifier Quality Measurement evaluates naming conventions. Identifier lengths are computed, and average length per file is derived. A threshold identifies poor naming. This helps improve readability and coding standards. Problem 3: Left Recursion Detection identifies grammar rules where the left-hand side appears at the start of the right-hand side. Such recursion causes issues in top-down parsing. A flag is generated to detect these cases. Problem 4: FIRST and FOLLOW overlap analysis checks parsing conflicts. Intersection of sets is computed, and overlap count is derived. If overlap exists, LL(1) violation is flagged. Problem 5: Parsing Table Density calculates how filled a parsing table is. Higher density indicates complexity. A threshold is applied to flag complex tables. Problem 6: Shift-Reduce Conflict Detection analyzes parser logs. Conflicts per state are counted, and states with multiple conflicts are flagged as unstable. Problem 7: LR(0) State Explosion Detection measures average items per state. High values indicate scalability issues and inefficient parsing. Problem 8: Temporary Variable Growth Analysis evaluates intermediate code. Number of temporary variables per block is calculated. Excessive temporaries indicate inefficiency. Problem 9: DAG Optimization identifies repeated expressions. Frequency of expressions is computed, and common subexpressions are flagged for optimization. Problem 10: Stack Growth Risk Analysis evaluates runtime memory usage. Stack frame size is calculated, and functions exceeding limits are flagged as risk. Conclusion: This assessment demonstrates how raw compiler data can be transformed into meaningful metrics. Using Python and Pandas, various indicators were designed to measure complexity, detect inefficiencies, and improve code quality. These techniques enhance understanding of compiler behavior and support better software design decisions.